

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

SEMINAR

Analiza bit-flip napada na velike jezične modele

Jakov Lovaković

Voditelj: prof.dr.sc. Domagoj Jakobović

Zagreb, svibanj, 2026

Sadržaj

1. Uvod.....	0
2. Bit-flip napadi.....	1
3. Bit-flip napadi na velike jezične modele.....	2
4. Programska potpora.....	3
4.1 Algoritam pretraživanja.....	3
4.2 Parametri algoritma pretraživanja.....	4
4.3 Pokretanje programske potpore.....	4
5. Rezultati.....	6
5.1 Pokus 1.....	6
5.2 Pokus 2.....	6
5.3 Pokus 3.....	6
5.4 Pokus 4.....	7
6. Zaključak.....	8
7. Literatura.....	9
8. Sažetak.....	10

1. Uvod

U ovome radu opisat će se ostvarenje programske potpore koja je bila izrađena u svrhu provedbe istraživanja utjecaja bit-flip napada na degradaciju performansi modela.

2. Bit-flip napadi

Bit-flip napadi temelje se na promjeni vrijednosti jednog ili više bitova u memoriji sustava, pri čemu dolazi do njihovog „preokretanja” (engl. *flip*).

Ovisno o tipovima podataka koji su pohranjeni u memoriji, preokretanje jednog ili više bitova može dovesti do značajne promjene vrijednosti podatka koji je pohranjen u memoriji.

Kao primjer možemo uzeti broj 3 zapisan u 32-bitnom IEEE-754 formatu. Njegov binarni zapis iznosi 01000000010000000000000000000000. Ako izmijenimo jedan bit dobivamo novi binaran zapis 01100000010000000000000000000000. Nova vrijednost iznosit će 5.5340232e19, što pokazuje kako jedna izmjena bita može značajno utjecati na vrijednost podatka.

3. Bit-flip napadi na velike jezične modele

Bit-flip napadi na velike jezične modele mogu se provesti na različite načine. Jedan oblik napada temelji se na izravnoj promjeni bitova težina transformera. Veliki jezični modeli temelje se na arhitekturi transformera, stoga izmjenom težina transformera možemo direktno utjecati na performanse modela.

Kao što je spomenuto u 2. poglavlju, izmjena vrijednosti jednog bita može značajno promijeniti performanse modela. Dodatno, ovisno o poziciji bita unutar težine transformera, promjena vrijednosti težine biti znatno izraženija pri izmjeni bitova eksponenta nego, primjerice, bitova mantise. Stoga u nastavku istraživanja, u svrhu testiranja robusnosti modela, odlučeno dodavati Gaussov šum na težine umjesto izravnog mijenjanja bita. Ovime se simulira izmjene bitova te se postižu bolji rezultati povezani s robusnošću modela.

4. Programska potpora

U svrhu pribavljanja rezultata ovoga istraživanja, napisana je programska podrška. Detaljni opis programskoga koda može se pronaći u bilježnici na repozitoriju.

Osnovna ideja algoritma jest pretraživanje težina jednoga sloja transformera te proučavanje degradacije prilikom izmjene tih težina.

U samome programu može se odabrati koji će se sloj pretraživati izmjenom funkcije `get_target_layer(model)`. U drugoj ćeliji se također može odabrati željeni model, promjenom `MODEL_NAME` varijable. Novi upiti za testiranje mogu se dodati u obliku para upit-odgovor u varijablu `QA_PAIRS` u trećoj ćeliji.

4.1 Algoritam pretraživanja

U svrhu pretrage razvijen je nasumičan algoritam pretraživanja. Pseudokod algoritma dan je u nastavku.

Algorithm 1 Pretraga najgore perturbacije težina

```
1: function SEARCHWORSTPERTURBATION(model, tokenizer, sentences, device)
2:   layer  $\leftarrow$  GetTargetLayer(model)
3:   inv  $\leftarrow$  GetWeightInventory(layer)
4:   baselineLoss  $\leftarrow$  EvaluateModel(model, tokenizer, sentences, device)
5:   worstLoss  $\leftarrow$  baselineLoss
6:   worstSelections  $\leftarrow$ 
7:   worstOriginals  $\leftarrow$ 
8:   itersSinceImprove  $\leftarrow$  0
9:   strategy  $\leftarrow$  "random"
10:  topPool  $\leftarrow$  initialTopPool
11:  for it = 1 to nIterations do
12:    if strategy = "random" then
13:      selections  $\leftarrow$  RandomSelection(inv, k)
14:    else
15:      selections  $\leftarrow$  MagnitudeSelection(inv, k, topPool)
16:    end if
17:    backups  $\leftarrow$  ApplyPerturbation(layer, selections, noiseStd)
18:    loss  $\leftarrow$  EvaluateModel(model, tokenizer, sentences, device)
19:    RestoreWeights(layer, backups)
20:    if loss > worstLoss then
21:      worstLoss  $\leftarrow$  loss
22:      worstSelections  $\leftarrow$  selections
23:      worstOriginals  $\leftarrow$  backups
24:      itersSinceImprove  $\leftarrow$  0
25:    else
26:      itersSinceImprove  $\leftarrow$  itersSinceImprove + 1
27:    end if
28:    if itersSinceImprove  $\geq$  patience then
29:      if strategy = "random" then
30:        strategy  $\leftarrow$  "magnitude"
31:        itersSinceImprove  $\leftarrow$  0
32:      else
33:        newPool  $\leftarrow$  max(50, topPool/2)
34:        if newPool  $\neq$  topPool then
35:          topPool  $\leftarrow$  newPool
36:          itersSinceImprove  $\leftarrow$  0
37:        else
38:          strategy  $\leftarrow$  "random"
39:          itersSinceImprove  $\leftarrow$  0
40:        end if
41:      end if
42:    end if
43:  end for
44:  return baselineLoss, worstLoss, worstSelections, worstOriginals
45: end function
```

4.2 Parametri algoritma pretraživanja

Algoritmu se mogu zadati sljedeći parametri:

- `n_iterations` – maksimalan broj iteracija
- `k` – koliko težina se bira u sloju
- `noise_std` – iznos standardne devijacije Gaussovog šuma
- `patience` – koliko iteracija treba proći bez boljeg rezultata da bi došlo do promjene strategije

4.3 Pokretanje programske potpore

Pokretanje je jednostavno. Preuzme se github repozitorij s <https://github.com/jakovlovakovic/transformer-weight-sensitivity/tree/main>. Locira se `.ipynb` bilježnica unutar `src` direktorija. `.ipynb` bilježnica se pokrene u bilo kojem online servisu, npr. „Google Colab”.

Primjer pokretanja:

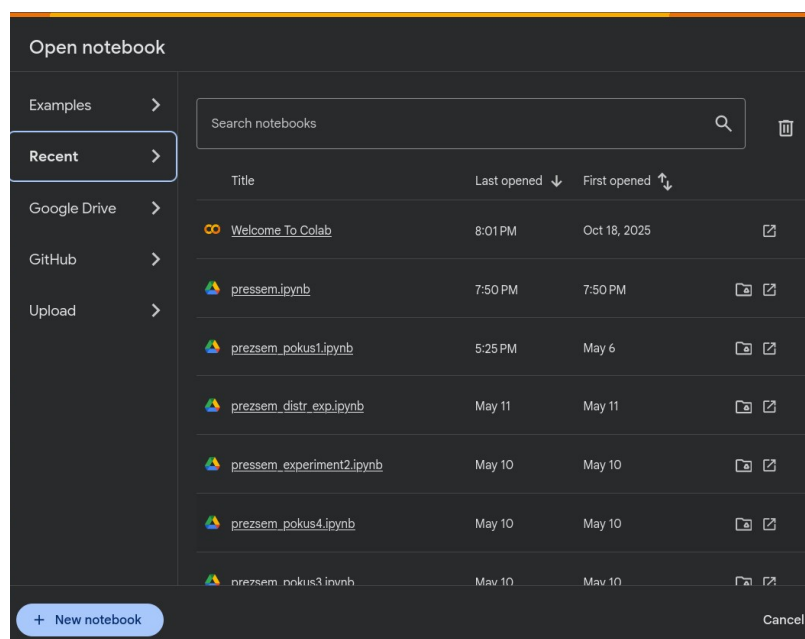


Figure 1: Odabрати opciju „Upload”

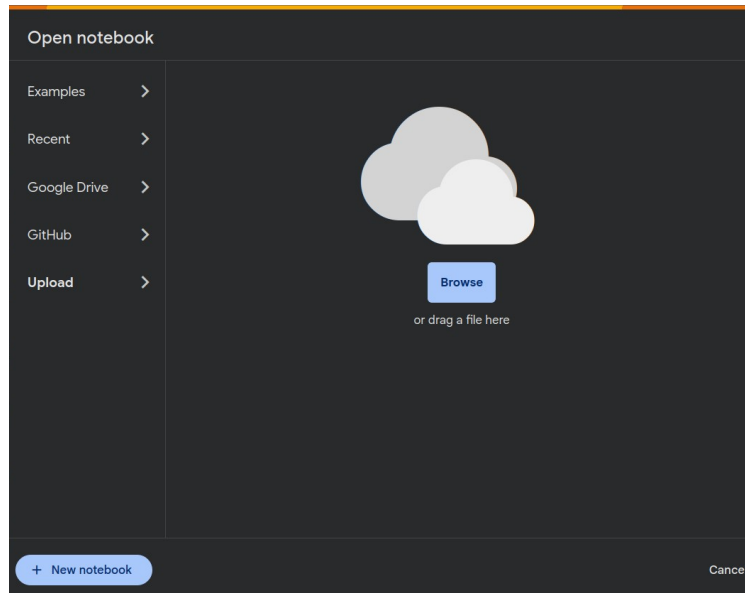


Figure 2: Odabрати preuzetu .ipynb datoteku

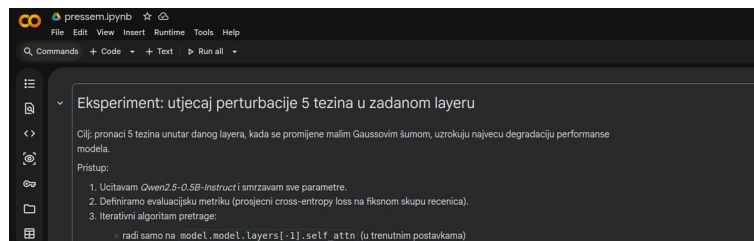


Figure 3: Kada je datoteka učitana, odabрати „Run all” opciju

5. Rezultati

U ovome poglavlju bit će opisani rezultati kroz više pokusa.

5.1 Pokus 1

U prvome pokusu proučavao se sloj pažnje transformera (engl. *attention layer*). U svrhu pretrage rezultat pretraga se radila na sloju pažnje u zadnjem bloku transformera.

Postavke algoritma pretraživanja su sljedeće:

- `n_iterations` = 200
- `k` = 5
- `noise_std` = 0.5
- `patience` = 30
- `initial_top_pool` = 1000

Rezultati su ukupna degradacija od 24.12% kada se promatra gubitak modela. To nije velika degradacija (pošto se gubitak promatra kao logaritam) te se pokazalo da je model i dalje siguran u svoje odluke. Početni gubitak iznosio je 0.109789, dok je rezultatni gubitak iznosio 0.136266.

5.2 Pokus 2

U drugome pokusu proučavao se sloj pažnje transformera (engl. *attention layer*). U svrhu pretrage rezultat pretraga se radila na sloju pažnje u zadnjem bloku transformera.

Postavke algoritma pretraživanja su sljedeće:

- `n_iterations` = 200
- `k` = 5
- `noise_std` = 0.5
- `patience` = 30
- `initial_top_pool` = 2000

Rezultati su ukupna degradacija od 28.16% kada se promatra gubitak modela. Početni gubitak iznosio je 0.109789, dok je rezultatni gubitak iznosio 0.140704. Prilikom povećanja vrijednosti varijable `initial_top_pool` mogu se uočiti i nešto bolji rezultati pokusa.

5.3 Pokus 3

U trećem pokusu proučavao se višeslojni perceptron (engl. *MLP*). U svrhu pretrage rezultat pretraga se radila na sloju pažnje u zadnjem bloku transformera.

Postavke algoritma pretraživanja su sljedeće:

- `n_iterations` = 200
- `k` = 5
- `noise_std` = 0.5
- `patience` = 30
- `initial_top_pool` = 5000

Rezultati su ukupna degradacija od 44.05% kada se promatra gubitak modela . Početni gubitak iznosio je 0.109789, dok je rezultatni gubitak iznosio 0.158148. Ovdje se može uočiti veća degradacija nego u prva dva pokusa.

5.4 Pokus 4

U trećem pokusu proučavao se višeslojni perceptron (engl. *MLP*). U svrhu pretrage rezultat pretraga se radila na sloju pažnje u zadnjem bloku transformera.

Postavke algoritma pretraživanja su sljedeće:

- `n_iterations` = 200
- `k` = 5
- `noise_std` = 0.5
- `patience` = 30
- `initial_top_pool` = 10000

Rezultati su ukupna degradacija od 251.57% kada se promatra gubitak modela . Početni gubitak iznosio je 0.109789, dok je rezultatni gubitak iznosio 0.385986. Ovdje se može uočiti najveća degradacija.

6. Zaključak

Najveća ranjivost uočena je kod višeslojnog perceptrona, gdje je degradacija dostigla 251.57%. Jedno od mogućih objašnjenja jest da MLP slojevi pohranjuju dio činjeničnog znanja modela, pa je algoritam pronašao težine čijom se izmjenom narušava to znanje. Zbog toga model za određene dijelove upita ne može donositi pouzdane zaključke.

Konkretno, u posljednjem je pokusu uočeno da model i dalje uspješno predviđa većinu upita, s iznimkom posljednjega. Riječ je o upitu „What is the largest planet in our solar system? Reply with one word.\nAnswer:”, na koji je točan odgovor „Jupiter”. Prije izmjene težina model je bio 65.98% siguran u svoj odgovor, dok je nakon izmjene razina sigurnosti pala svega na 7.22%.

Ovi rezultati sugeriraju da je višeslojni perceptron više ranjiv na ovakve izmjene težina nego sloj pažnje, ali da bi se ovo praktično utvrdilo potrebno je pokrenut veći broj pokusa na više blokova transformera.

Moguća nadogradnja je korištenje iterativnog algoritma pretraživanja koji koristi informacije iz prethodne iteracije kako bi generirao novo rješenje.

7. Literatura

Sanjay Das, Swastik Bhattacharya, Souvik Kundu, Shamik Kundu, Anand Menon, Arnab Raha, Kanad Basu. GenBFA: An Evolutionary Optimization Approach to Bit-Flip Attacks on LLMs. 2024. <https://arxiv.org/abs/2411.13757>

8. Sažetak

U ovome radu razmatran je utjecaj izmjene težina na performanse velikih jezičnih modela temeljenih na arhitekturi transformera. Dana je programska potpora za pomoću pri pretraživanju ranjivih težina. Bit-flip napadi temelje se na prevrtanju (engl. flip) jednog ili više bitova u memoriji sustava, pri čemu i mala promjena može značajno utjecati na vrijednost pohranjenih podataka. U radu je analiziran utjecaj izmjene težina transformera dodavanjem Gaussovog šuma na težine modela, čime se simulira promjena bitova u memoriji.

Provedeno je više pokusa na slojevima pažnje i višeslojnim perceptronima. Rezultati pokazuju da su višeslojni perceptroni osjetljiviji na izmjene težina od slojeva pažnje.

Ključne riječi: bit-flip napadi, veliki jezični modeli, transformeri, sloj pažnje, višeslojni perceptron, Gaussov šum, nasumičan algoritam pretraživanja